Software Requirements Specification

Restaurant App MVP (Frontend Only, React Native)

Version 1.0

Fall 2026

Course: Mobile Application Development

Assignment 1

Prepared by: Manahil Fatima

Reg no: 9712

1. Introduction

1.1 Purpose

This document defines the requirements of the Restaurant App MVP version 1.0. The app lets diners browse a menu, search and filter dishes, build a cart, reserve a table, place an order (dine-in or takeaway) and track it. It also lets a restaurant manager manage the menu, reservations and incoming orders. The intended audience is the course instructor and TAs (evaluation), the developer (implementation) and future maintainers who will extend the app with a backend. This version covers a frontend-only prototype in which all data is mock data held in React state.

1.2 Scope

In scope (version 1.0):

- Login and Signup with role selection (Customer or Manager) against a mock users array.
- Menu browsing by category with Daily Special badges and unavailable-item handling.
- Search with debounce, recent search suggestions, sorting and a Back to top control.
- Cart with quantity stepper, per-item special instructions and promo codes (WELCOME10, FEAST20).
- Order Summary with subtotal, 5% service charge, 15% sales tax, discount and grand total.
- Table reservation with time-slot availability, validation, confirmation modal and cancellation.
- Order placement (Dine-in with table, Takeaway with pickup time) and simulated Order Tracking.
- Profile screen with light and dark theme toggle and role-based navigation.
- Manager Dashboard: incoming orders, reservation accept or decline, menu management.
- Local persistence of orders, reservations and menu edits using AsyncStorage.

Out of scope: real payment processing, push notifications, any backend service or database, external APIs, real authentication security (password hashing, tokens), image upload, multi-restaurant support and any state management library such as Redux.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification: this document, describing what the system must do.
MVP	Minimum Viable Product: the smallest version of the app that delivers its core value.
UML	Unified Modeling Language: a standard notation for diagrams that model software.
Hook	A React function (useState, useEffect and so on) that lets function components use state and lifecycle features.
Custom Hook	A user-written function starting with 'use' that packages reusable hook logic (for example useForm).
Context	React's Context API, used to share state across components without prop drilling.
Reducer	A pure function (state, action) that returns the next state; used with useReducer.
Mock Data	Fake sample data stored in local JS files that stands in for a database.
FlatList	A React Native component that renders long lists efficiently by drawing only visible rows.
AsyncStorage	A key-value store on the device used to persist data between app launches.
Debounce	Delaying an action until the user has stopped typing for a set time (400 ms here).
Expo Go	A mobile app used to run React Native projects during development without a native build.

2. Overall Description

2.1 User Roles

Role	Key Permissions and Goals
Customer	Sign up and log in; browse, search, sort and favourite menu items; manage cart and apply

Role	Key Permissions and Goals
	promo codes; reserve and cancel tables; place dine-in or takeaway orders; track order status; change theme in Profile. Goal: order quickly and avoid queues.
Restaurant Manager	Log in to a role-restricted Dashboard; view and update order status; accept or decline reservations; add items, edit prices and toggle availability. Goal: digitize the menu and handle bookings and orders efficiently.

2.2 User Stories

Self-interview summary (customer view). Questions I asked myself: What annoys me at restaurants? What would make me download this app? Findings: (1) waiting for a menu and waiter, (2) not knowing if a table is free at 7 PM, (3) not knowing what is sold out or special today, (4) wanting to customize a dish such as 'no onions', (5) wanting to know how long my food will take and when it is ready, (6) wanting a dark mode for evening use. These feed the stories marked (Interview) below.

ID	User Story
US-01 (Interview)	As a customer, I want to browse the menu by category, so that I can find food without waiting for a waiter.
US-02 (Interview)	As a customer, I want to see Daily Special badges, so that I can pick the best offers quickly.
US-03 (Interview)	As a customer, I want sold-out items to be greyed out, so that I do not order something unavailable.
US-04 (Interview)	As a customer, I want to book a table for a chosen time and see which slots are free, so that I do not arrive and find no seat.
US-05 (Interview)	As a customer, I want to add notes like 'no onions' to a cart item, so that my food matches my taste.
US-06 (Interview)	As a customer, I want to track my order status live, so that I know when my food is ready.
US-07	As a customer, I want to search dishes by name, so that I can find a favourite item fast.

ID	User Story
US-08	As a customer, I want to apply a promo code and see the full price breakdown, so that I know exactly what I will pay.
US-09	As a restaurant manager, I want to add items, edit prices and toggle availability, so that the menu is always accurate.
US-10	As a restaurant manager, I want to see incoming orders and update their status, so that customers are informed.
US-11	As a restaurant manager, I want to accept or decline reservations, so that I can manage seating.
US-12	As a customer, I want a dark mode, so that the app is comfortable to use at night.

3. Functional Requirements

Authentication

- FR-01: The system shall provide a screen that switches between Login and Signup modes.
- FR-02: The system shall validate that email is well formed, password has at least 8 characters including one digit, and password and confirm password match.
- FR-03: The system shall authenticate users against the mock users array and show an Alert on failure.
- FR-04: The system shall navigate a Customer to the Menu screen and a Manager to the Dashboard after login, and shall allow logout that resets the navigation stack.

Menu

- FR-05: The system shall display at least 15 menu items in at least four categories (Starters, Mains, Desserts, Drinks).
- FR-06: The system shall show a loading indicator while the menu loads and an error message with a Retry button if loading fails.
- FR-07: The system shall show a 'Daily Special' badge on items where isSpecial is true.
- FR-08: The system shall display unavailable items greyed out with the Add button disabled.

Search

- FR-09: The system shall filter menu items by name 400 ms after the user stops typing.
- FR-10: The system shall show the last five distinct search terms as tappable suggestions when the input is focused and empty.

- FR-11: The system shall sort items by price low to high, price high to low or name A to Z, and show an empty-state message when nothing matches.

Cart

- FR-12: The system shall let the customer add, remove, increment and decrement items, removing an item when its quantity reaches zero.
- FR-13: The system shall let the customer attach a special instruction to each cart item.
- FR-14: The system shall accept the promo codes WELCOME10 and FEAST20 and reject any other code with an error message.
- FR-15: The system shall show a live item-count badge on the cart tab icon.

Reservation

- FR-16: The system shall offer hourly slots from 12:00 to 22:00 and disable a slot when no table with enough seats is free.
- FR-17: The system shall reject past dates, party sizes outside 1 to 12, phone numbers not in the format 03XX-XXXXXXX and bookings less than one hour ahead.
- FR-18: The system shall show a confirmation modal summarizing the booking before saving it.
- FR-19: The system shall list the user's reservations and allow cancellation after an Alert confirmation.

Orders

- FR-20: The system shall calculate subtotal, 5% service charge, 15% sales tax, discount and grand total on the Order Summary screen.
- FR-21: The system shall let the customer choose Dine-in (with a table) or Takeaway (with a pickup time) when placing an order.
- FR-22: The system shall create each order with id, items, total, type, status 'Pending' and a timestamp.
- FR-23: The system shall move an order automatically to Preparing after 10 s, Ready after 20 s and Served after 30 s, and display a step progress indicator and an elapsed-time counter updating every second.

Manager Dashboard

- FR-24: The system shall show the Dashboard only to users whose role is manager.
- FR-25: The system shall let the manager change an order's status manually from the Incoming Orders tab.
- FR-26: The system shall let the manager accept or decline reservations from the Reservations tab.
- FR-27: The system shall let the manager add an item, edit a price and toggle availability, and shall show these changes on the customer menu immediately.
- FR-28: The system shall persist orders, reservations and menu edits in AsyncStorage and reload them at startup behind a loading screen.

4. Non-Functional Requirements

Category	Requirement
Usability	Maximum three taps to add an item to the cart; clear Alerts and inline field errors; error text clears as soon as the user edits the field; touch targets of at least 44 px.
Performance	Each screen shall render within 1 second of data being available; menu lists use FlatList so scrolling stays smooth with 15+ items; derived data uses useMemo and cards use React.memo so that only changed cards re-render.
Responsiveness	Layouts use flexbox and percentage widths and safe-area insets so they work on small and large phones in portrait.
Maintainability	Source is organized under src/components, screens, context, reducers, hooks, data and navigation; business logic lives in custom hooks and reducers, not in screens; shared theme file holds all colours.
Data handling without backend	All data comes from local mock files, is held in React state and persisted in AsyncStorage; the app shall run with no network, backend, external API or state management library.
Reliability	Timers and intervals are cleared on unmount so no state update happens on an unmounted screen; the app never shows empty data while loading.
Theming	Every screen reads colours from useTheme so toggling dark mode updates the whole app instantly.

5. Client-Side Data Model (Mock Data)

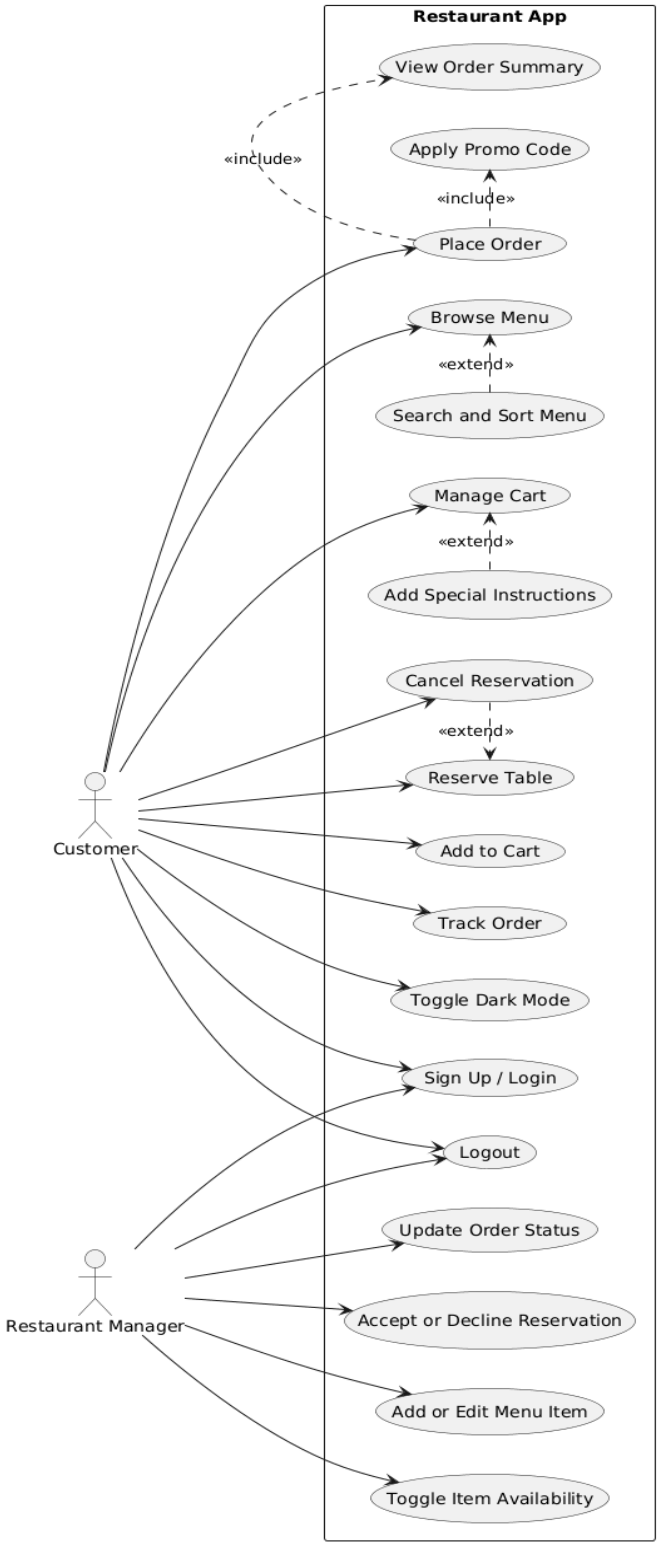
Data Set Name	Fields	Description
Users	id, fullName, email, password, role ('customer' or 'manager')	Mock accounts in src/data/users.js; includes one customer and one manager for login.

Data Set Name	Fields	Description
Categories	id, name	Starters, Mains, Desserts, Drinks; used for category chips.
MenuItems	id, name, description, price, category, image, isSpecial, isAvailable	At least 15 items in src/data/menu.js; edited by the manager and persisted.
Tables	id, number, seats	At least six tables with different seating capacities, used for availability checks.
Reservations	id, userId, tableId, date, timeSlot, partySize, phone, status ('pending', 'accepted', 'declined', 'cancelled')	Bookings made by customers; checked against Tables for clashes.
Orders	id, userId, items [{itemId, name, price, qty, note}], total, type ('dine-in' or 'takeaway'), tableId or pickupTime, status, createdAt	Orders created from the cart; status moves through the lifecycle.
PromoCodes	code, discountPercent	Local object: WELCOME10 = 10, FEAST20 = 20.

6. UML Diagrams (Placeholders)

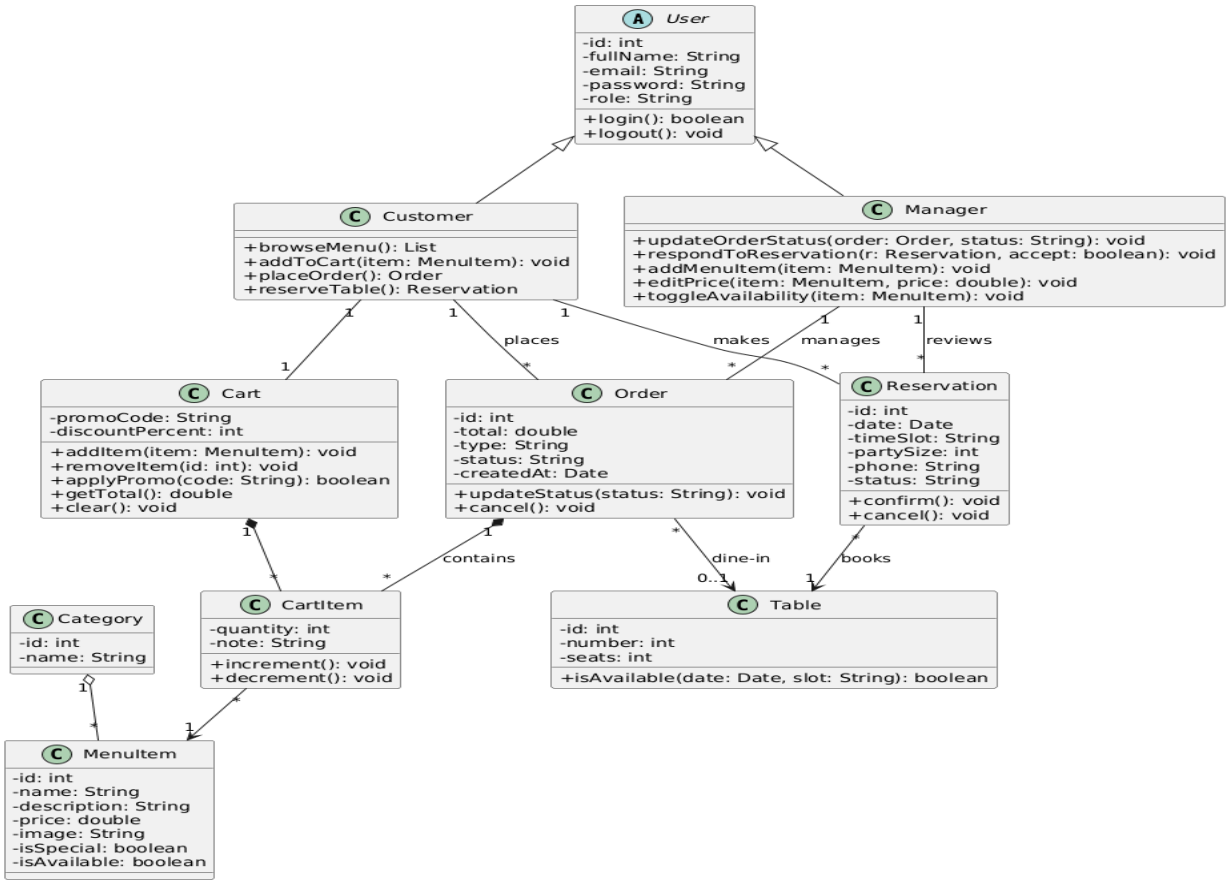
Each diagram below will be inserted after Question 2 is completed.

Figure 1: Use Case Diagram.



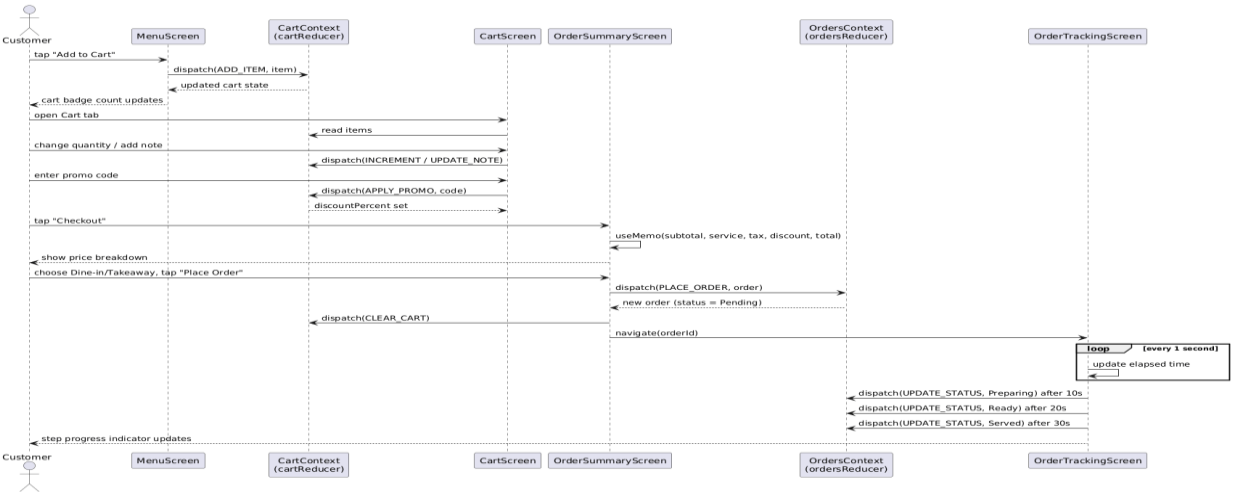
Shows the Customer and Restaurant Manager actors with at least twelve use cases, using include and extend relationships such as Place Order including Apply Promo Code. It gives a quick overview of what each role can do.

Figure 2: Class Diagram.



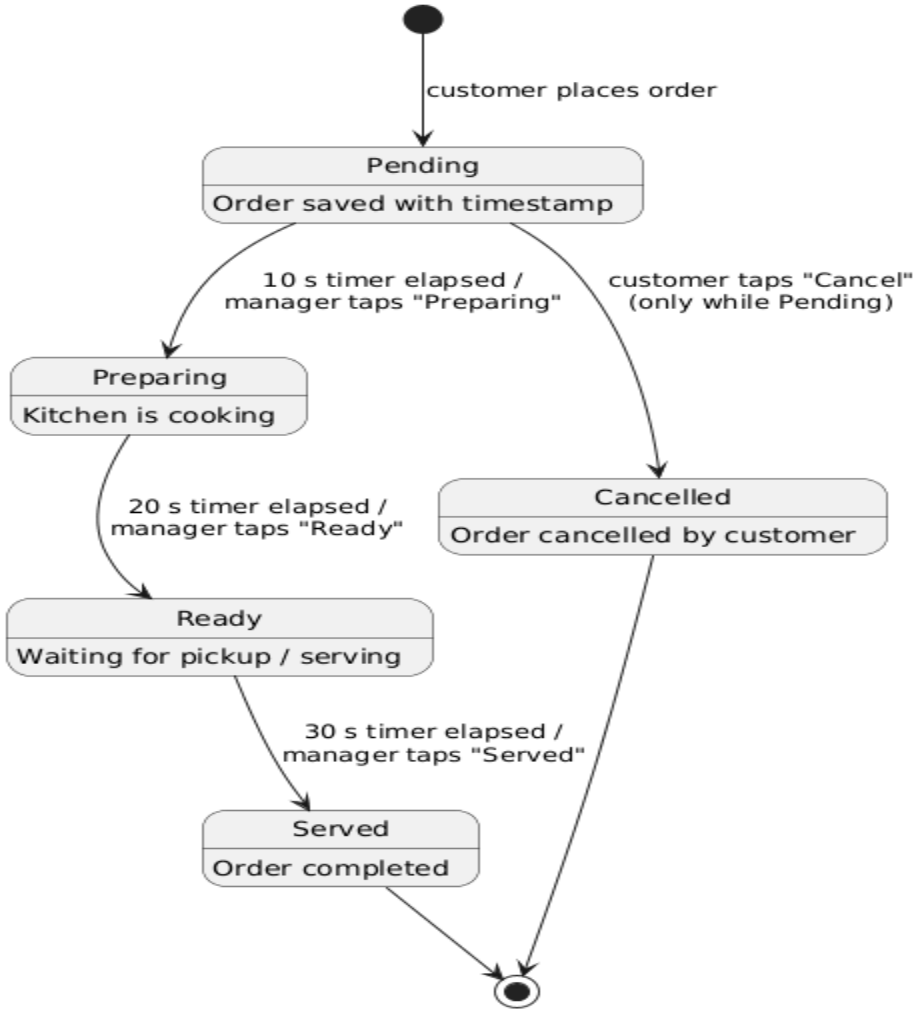
Shows User with its subclasses Customer and Manager, and the classes Category, MenuItem, Cart, CartItem, Order, Reservation and Table, with attributes, operations and multiplicities. It defines the static structure of the data.

Figure 3: Sequence Diagram: Customer Places an Order.



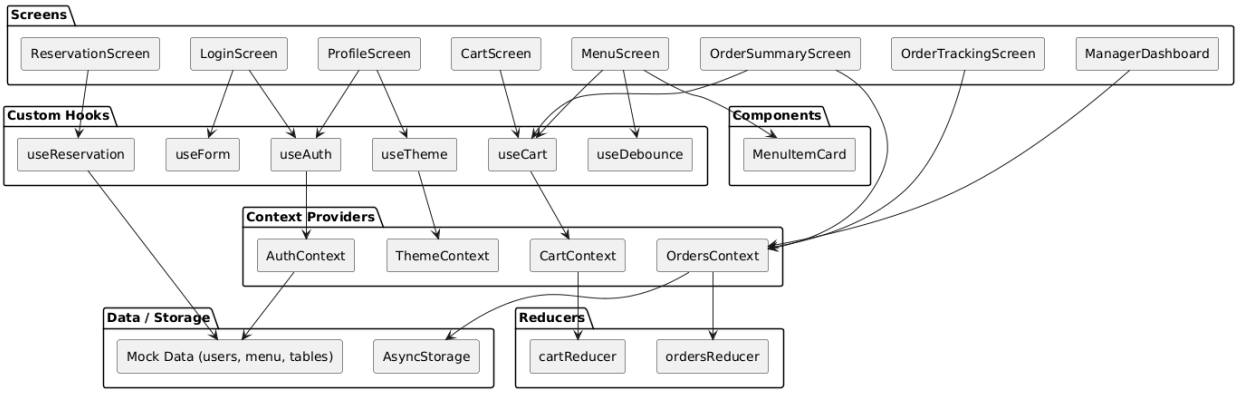
Shows the message flow from tapping Add to Cart through the cart reducer, order summary and OrdersContext to the Order Tracking screen.

Figure 4: State Machine Diagram: Order Lifecycle.



Shows the states Pending, Preparing, Ready, Served and Cancelled and the events (timer elapsed, manager action, customer cancel) that cause each transition.

Figure 5: Component Diagram.



Shows screens, context providers, reducers and custom hooks and how they depend on each other.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

- **Purpose:** Authenticate users and route them by role.
- **UI elements:** Mode toggle (Login or Signup), inputs for full name, email, password, confirm password, role selector, show-password toggle, submit button with ActivityIndicator, inline error text.
- **Navigation:** Entry: app start (when no user). Exit: Customer goes to Menu, Manager goes to Dashboard.
- **Local data:** src/data/users.js.
- **Hooks:** useState (mode, field values, errors, showPassword, isSubmitting) for controlled form state; later useForm (custom) for reusable form logic and useAuth (useContext) to store the user globally.

7.2 Menu Browsing Screen

- **Purpose:** Let customers browse available dishes.
- **UI elements:** Horizontal category chips, FlatList of item cards, Daily Special badge, greyed-out unavailable cards, Add button, loading indicator, error message with Retry, pull to refresh.
- **Navigation:** Entry: after customer login or Menu tab. Exit: Cart tab, Profile tab.
- **Local data:** src/data/menu.js.
- **Hooks:** useState (menuItems, isLoading, error, selectedCategory, refreshing); useEffect (simulated 1.5 s fetch with timer cleanup, and header title update); later useMemo for derived filtered list.

7.3 Search and Scroll Controls

- **Purpose:** Help users find items fast in a long list.
- **UI elements:** Search bar with search icon and clear button, recent-search suggestions, Back to top floating button, sort options, empty-state message, render counter debug label.
- **Navigation:** Part of the Menu screen.
- **Local data:** menuItems, recent searches (state).

- **Hooks:** useRef (TextInput focus, debounce timer id, FlatList scrolling, render counter, previous query) because refs persist without re-rendering; useState (scroll offset visibility, recent searches); later custom useDebounce.

7.4 Profile and Theme Screen

- **Purpose:** Show account details and control theme.
- **UI elements:** Name, email and role display, dark mode switch, logout button.
- **Navigation:** Entry: Profile tab. Exit: logout returns to Login with reset stack.
- **Local data:** logged-in user from AuthContext, light and dark palettes in the theme file.
- **Hooks:** useContext through useAuth and useTheme so that the whole app shares user and theme without prop drilling.

7.5 Cart Screen

- **Purpose:** Review and edit selected items.
- **UI elements:** Item rows with quantity stepper, remove button, special-instructions input, promo code field with error text, item count badge on tab.
- **Navigation:** Entry: Cart tab. Exit: Order Summary or back to Menu.
- **Local data:** cart state { items, promoCode, discountPercent } and promo code object.
- **Hooks:** useReducer (cartReducer with ADD_ITEM, REMOVE_ITEM, INCREMENT, DECREMENT, UPDATE_NOTE, CLEAR_CART, APPLY_PROMO, REMOVE_PROMO) because many related transitions need one pure update function; useContext via CartContext and useCart.

7.6 Order Summary Screen

- **Purpose:** Show the final price breakdown before ordering.
- **UI elements:** Line items, subtotal, service charge, tax, discount, grand total, Place Order button.
- **Navigation:** Entry: Cart screen. Exit: Order Tracking after placing.
- **Local data:** cart state and constants SERVICE_RATE = 0.05, TAX_RATE = 0.15.
- **Hooks:** useMemo (totals depend only on items and discount); useCallback and React.memo on MenuItemCard so handlers stay stable and only the toggled card re-renders.

7.7 Table Reservation Screen

- **Purpose:** Let customers book and cancel tables.
- **UI elements:** Date picker, hourly slot grid (disabled when full), party size input, table selector, phone input, confirmation modal, My Reservations list with cancel.
- **Navigation:** Entry: Reservations tab. Exit: back to Menu after booking.
- **Local data:** mockTables, mockReservations.
- **Hooks:** custom useReservation (all availability and validation logic), useForm and useDebounce; the screen holds only UI code.

7.8 Order Tracking and Manager Dashboard

- **Purpose:** Let customers watch order progress and let managers run the restaurant.

- **UI elements:** Step progress indicator, elapsed timer; Dashboard with three tabs: Incoming Orders (status buttons), Reservations (accept or decline), Menu Management (add item, edit price, availability switch); loading screen at startup.
- **Navigation:** Entry: after placing order (customer) or Dashboard tab (manager only). Exit: tabs and logout.
- **Local data:** OrdersContext, reservations, menu shared state, AsyncStorage.
- **Hooks:** useEffect with setInterval for status changes and elapsed time, with cleanup; useEffect for AsyncStorage load and save; useReducer for OrdersContext; useContext so manager menu edits reach the customer menu immediately.